

## MEMBER 3

## ML Engineer

PySpark MLlib delay classifier (optional extension)

Name: \_\_\_\_\_

The shared table contract, metric definitions, risks and timeline are in **00\_Team\_Overview**. Read it first. This brief only covers your role.

## YOUR FIRST 3 HOURS

- Hour 1: agree the contract with the team.** Check Table 8, `ml_delay_predictions` (its DDL is below).
- Fix the label:** `label = ArrDel15`. Question: will the flight arrive  $\geq 15$  min late? Use **operated flights only**.
- Fix the allowed features** (8 columns, in the ML rules below). Learn the forbidden list: anything not known before departure is **data leakage**.
- Fix the split:** train on 2022–2023 (a 10–20% random sample), test on 2024.

Your work is an optional extension. If the team falls behind, ML and Power BI page 4 are cut first (see 00\_Team\_Overview).

## Your deliverables

- Leak-free delay classifier (label `ArrDel15`)
- AUC-ROC, precision / recall, confusion matrix
- Feature importance / coefficients
- `ml_delay_predictions` loaded into ClickHouse ( $\leq 500k$  rows)
- ROC curve and feature-importance visuals for the deck
- A short live demo / terminal script for Q&A
- Content for slides 9 and 10

## Definition of done

- ☐ Label, allowed features and split fixed (hours 0–3).
- ☐ Feature pipeline uses `StringIndexer` (`handleInvalid="keep"`), `OneHotEncoder` (not `OneHotEncoderEstimator`) and `VectorAssembler`, with allowed features only.
- ☐ Population is operated flights only.
- ☐ Baseline `LogisticRegression` built. `RandomForestClassifier` is optional ( $\leq 30$  trees, depth  $\leq 8$ ).
- ☐ Trained on a 10–20% sample of 2022–2023. Evaluated on 2024. AUC-ROC reported. About 0.60–0.70 is expected. Much higher means leakage.
- ☐ Precision, recall and confusion matrix documented. Accuracy alone is not enough.
- ☐ Coefficients / feature importance and top drivers documented.
- ☐ `ml_delay_predictions` loaded with  $\leq 500k$  rows. `Risk_Category` set: Low  $< 0.15$ , Medium 0.15–0.30, High  $\geq 0.30$  (adjust after viewing the distribution).
- ☐ ROC curve and feature-importance visuals ready for the deck.
- ☐ Short demo / terminal script ready for Q&A.
- ☐ Content ready for slides 9 and 10.

## Your schedule, hour by hour

Hours count from kickoff. Text is taken from the plan's schedule tables.

| When                                                                 | What you do (from the plan)                                                                                                                     |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 1 — foundation, parallel work on the 1-month sample (hours 0–24) |                                                                                                                                                 |
| Hours 0–3                                                            | Fix label ( <code>ArrDel15</code> ), allowed features, train / test split (see the ML rules below).                                             |
| Hours 3–8                                                            | Feature pipeline on the sample: <code>StringIndexer</code> , <code>OneHotEncoder</code> , <code>VectorAssembler</code> (allowed features only). |
| Hours 8–14                                                           | Baseline <code>LogisticRegression</code> (optional <code>RandomForest</code> ) on a sample. Compute AUC.                                        |

| When                                                               | What you do (from the plan)                                                        |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Hours 14–20                                                        | Coefficients / feature importance. Build predictions sample ( $\leq 500k$ rows).   |
| Hours 20–24                                                        | Load predictions into <code>ml_delay_predictions</code> .                          |
| Day 2 — full 48-month run, polish, defense readiness (hours 24–48) |                                                                                    |
| Hours 24–30                                                        | Train on 10–20% sample of 2022–2023, evaluate on 2024; record AUC-ROC and metrics. |
| Hours 30–36                                                        | Document confusion matrix, precision / recall, top drivers.                        |
| Hours 36–42                                                        | ROC curve and feature-importance visuals for the deck.                             |
| Hours 42–48                                                        | Prepare a short live demo / terminal script for Q&A.                               |

## Handoffs: who sends what, by when

| You receive from | What                                                                                      | By when                                    |
|------------------|-------------------------------------------------------------------------------------------|--------------------------------------------|
| All members      | The agreed contract, especially Table 8                                                   | Hour 1                                     |
| Member 2         | Cleaned Parquet at <code>/project/flights/parquet/</code> (sample first, full data later) | Sample: hours 3–8 †<br>Full: hours 24–30 † |
| Member 1         | Table <code>ml_delay_predictions</code> , created from the DDL                            | Hours 14–20 †                              |

| You send to | What                                                                          | By when                                      |
|-------------|-------------------------------------------------------------------------------|----------------------------------------------|
| Member 4    | Predictions loaded into <code>ml_delay_predictions</code> ( $\leq 500k$ rows) | Hours 20–24                                  |
| Member 1    | The training script, for the <code>train_model</code> task in the Airflow DAG | Hours 20–24 †                                |
| Member 4    | AUC-ROC, confusion matrix, precision / recall, top drivers                    | Hours 30–36 †                                |
| Member 4    | ROC curve and feature-importance visuals for the deck                         | Hours 36–42                                  |
| Member 4    | Content for slides 9 and 10                                                   | Before slides 7–12 are built (hours 36–42) † |

† Not written as a handoff in the plan, but the schedule needs it. Agree the exact hour with the other member.

## ML rules

Copy from the project context. Follow it exactly.

| Item                              | Rule                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Task                              | Binary classification: will the flight arrive $\geq 15$ min late?                                                                                                                                                                                                                                                                             |
| Label                             | <code>label = ArrDel15</code>                                                                                                                                                                                                                                                                                                                 |
| Population                        | Operated flights only (drop cancelled, diverted, and null <code>ArrDel15</code> )                                                                                                                                                                                                                                                             |
| Allowed features                  | <code>Month</code> , <code>DayOfWeek</code> , <code>Dep_Hour</code> , <code>Reporting_Airline</code> , <code>Origin</code> , <code>Dest</code> , <code>Distance</code> , <code>CRSElapsedTime</code> (all known before departure)                                                                                                             |
| Forbidden features (data leakage) | <code>DepTime</code> , <code>DepDelay</code> , <code>DepDelayMinutes</code> , <code>DepDel15</code> , <code>ArrTime</code> , <code>ActualElapsedTime</code> , <code>AirTime</code> , <code>ArrDelay*</code> , all <code>*Delay</code> cause columns, <code>Cancelled</code> , <code>Diverted</code> , any taxi / wheels-off / wheels-on field |
| Split                             | Train on 2022–2023 (use a 10–20% random sample); test on 2024. Optional final check on 2025.                                                                                                                                                                                                                                                  |
| Models                            | <code>LogisticRegression</code> (primary, fast) and optionally <code>RandomForestClassifier</code> ( $\leq 30$ trees, depth $\leq 8$ )                                                                                                                                                                                                        |
| Spark 3 API                       | Use <code>OneHotEncoder</code> ( <b>not</b> <code>OneHotEncoderEstimator</code> , which only exists in Spark 2.x). <code>StringIndexer(handleInvalid="keep")</code> . If a tree model uses indexed categoricals directly, set <code>maxBins <math>\geq 400</math></code> (airports $\approx 350$ ).                                           |
| Metrics                           | AUC-ROC (main), precision, recall, confusion matrix. Accuracy alone is misleading, since roughly 80% of flights are on time.                                                                                                                                                                                                                  |
| Expected result                   | Modest: AUC around 0.60–0.70 is normal for schedule-only features. A much higher AUC means leakage.                                                                                                                                                                                                                                           |

| Item   | Rule                                                                                                                                                                                                 |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Output | <code>ml_delay_predictions</code> , a ≤ 500k-row sample of the test set. Risk_Category thresholds on probability: Low < 0.15, Medium 0.15–0.30, High ≥ 0.30 (adjust after viewing the distribution). |

### Metric definitions you need:

- **Total\_Flights** = all rows, including cancelled and diverted.
- **Operated flight** = `Cancelled = 0 AND Diverted = 0 AND ArrDel15 not null`.
- **Delayed\_Flights** = operated with `ArrDel15 = 1`; **OnTime\_Flights** = operated with `ArrDel15 = 0`.
- **Dep\_Hour** = `floor(CRSDepTime / 100) % 24`. **Day\_Of\_Week**: 1 = Monday ... 7 = Sunday (BTS `DayOfWeek`).

### Warnings for your role

- **Data leakage is the biggest risk.** Never use: `DepTime`, `DepDelay*`, `DepDel15`, `ArrTime`, `ActualElapsedTime`, `AirTime`, `ArrDelay*`, any delay-cause column, `Cancelled`, `Diverted`, or any taxi / wheels field.
- **A modest AUC is normal.** Schedule-only features give about 0.60–0.70. A much higher AUC means leakage. If the AUC looks low, report it honestly. Do **not** add leaking features.
- **Accuracy alone is misleading.** Roughly 80% of flights are on time. Report AUC-ROC (main), precision, recall and the confusion matrix.
- **Spark 3 API.** Use `OneHotEncoder`, not `OneHotEncoderEstimator` (that is Spark 2.x only). Use `StringIndexer(handleInvalid="keep")`.
- **Tree models.** If a tree model uses indexed `Origin` / `Dest` directly, set `maxBins ≥ 400` (about 350 airports). With one-hot features this is not needed.
- **Train on a 10–20% sample of 2022–2023** and test on 2024. Sample the test set first if it is slow.
- **Load at most 500k rows** into `ml_delay_predictions` (a sample of the 2024 test set), not millions.
- **The code below is a starter, not tested against your cluster.** Adjust paths and options. Run it on the 1-month sample first.
- **ML is optional.** It is the first thing cut if the team falls behind.

## Commands, DDL and code

**ML starter, leak-free (Python / PySpark). It ends with a DataFrame called `out`.**

```
from pyspark.sql import functions as F
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.functions import vector_to_array

df = spark.read.parquet("hdfs:///project/flights/parquet")
df = df.filter((F.col("Cancelled") == 0) & (F.col("Diverted") == 0) & F.col("ArrDel15").isNotNull())
df = df.withColumn("label", F.col("ArrDel15"))

cat = ["Reporting_Airline", "Origin", "Dest"]
num = ["Month", "DayOfWeek", "Dep_Hour", "Distance", "CRSElapsedTime"] # allowed features only

stages = [StringIndexer(inputCol=c, outputCol=c + "_idx", handleInvalid="keep") for c in cat]
stages += [OneHotEncoder(inputCols=[c + "_idx" for c in cat],
                           outputCols=[c + "_ohe" for c in cat], handleInvalid="keep")]
stages += [VectorAssembler(inputCols=[c + "_ohe" for c in cat] + num,
                           outputCol="features", handleInvalid="skip")]
stages += [LogisticRegression(featuresCol="features", labelCol="label", maxIter=20)]

train = df.filter(F.col("Year") <= 2023).sample(fraction=0.15, seed=42)
test = df.filter(F.col("Year") == 2024) # sample first if slow

model = Pipeline(stages=stages).fit(train)
pred = model.transform(test)

auc = BinaryClassificationEvaluator(labelCol="label", metricName="areaUnderROC").evaluate(pred)
print("AUC-ROC:", auc) # ~0.60-0.70 is expected; much higher = leakage

out = (pred.withColumn("p", vector_to_array("probability")[1])
       .select("FlightDate", "Reporting_Airline", "Origin", "Dest", "Dep_Hour",
              F.col("label").cast("int").alias("Actual_Delayed"),
              F.col("prediction").cast("int").alias("Predicted_Delayed"),
              F.col("p").cast("float").alias("Delay_Probability"))
       .withColumn("Risk_Category",
                  F.when(F.col("Delay_Probability") < 0.15, "Low")
                    .when(F.col("Delay_Probability") < 0.30, "Medium").otherwise("High"))
       .sample(fraction=0.07, seed=1).limit(50000)) # <= 500k rows for ClickHouse
```

### DDL for your output table (SQL)

```
CREATE TABLE IF NOT EXISTS ml_delay_predictions (
  FlightDate Date, Reporting_Airline String, Origin String, Dest String,
  Dep_Hour UInt8, Actual_Delayed UInt8, Predicted_Delayed UInt8,
  Delay_Probability Float32, Risk_Category String
) ENGINE = MergeTree() ORDER BY (FlightDate, Reporting_Airline);
```

**Load method: export to CSV (Python). Same method for `ml\_delay\_predictions`: change the table and file names.**

```
import os
os.makedirs("/tmp/out", exist_ok=True)
t1.toPandas().fillna(0).to_csv("/tmp/out/agg_airline_performance.csv", index=False) # repeat per table
```

### Load method: CSV → ClickHouse (bash)

```
clickhouse-client --query "INSERT INTO flight_analytics.agg_airline_performance FORMAT CSVWithNames" \
  < /tmp/out/agg_airline_performance.csv
# Docker: docker exec -i clickhouse clickhouse-client --query "..." < file.csv
```